Software Architecture and Design Specification

Project: Online Exam Portal (SecureExam-AI)

AI-Powered Secure Online Examination System with Smart Proctoring and Analytics

Field	Details
Project	Online Exam Portal (SecureExam-AI)
Document	Software Architecture and Design Specification (SAD)
Version	1.0
Authors / Team Members	Krishna H Zalavadiya, DEERAJ S, Kondapuram Charith Sai Reddy, Korada Sannath Kumar
Course Coordinator / Guide	Sheela Devi
Date	04-10-2026
Status	Draft — For Review
Reference Documents	SRS — Online Exam Portal (SecureExam-AI), v1.0, 06-09-2026; Software Test Plan (STP), v1.0, 16-09-2026

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Student Team	Initial SAD prepared from the approved project SRS, Software Test Plan, and provided SAD template.

Approvals

Role	Name	Signature / Date
Project Guide / Evaluator	Sheela Devi	
QA Lead	Korada Sannath Kumar	
Product Owner	Krishna H Zalavadiya	
Developer	DEERAJ S	
Test Engineer	Kondapuram Charith Sai Reddy	

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Online Exam Portal (SecureExam-AI). It translates the requirements established in the Software Requirements Specification into an architectural view covering major components, responsibilities, interfaces, data storage, security, design flows, and traceability.

1.2 Scope

The system covers Student registration and authentication, Admin exam and question management, timed fullscreen exam attempts, browser-based AI proctoring, automatic evaluation, result publication and history, Admin analytics, and Excel-based result export. Live human-invigilator video calling, payment/billing, and external university ERP integration remain outside the current version scope.

1.3 Audience

Developers, QA engineers, the Project Guide/Evaluator, System Administrators, security reviewers, and project team members responsible for maintenance and further development.

1.4 Definitions

SRS — Software Requirements Specification; STP — Software Test Plan; SAD — Software Architecture and Design Specification; RTM — Requirements Traceability Matrix; JWT — JSON Web Token; REST — Representational State Transfer; RBAC — Role-Based Access Control; ODM — Object Document Mapper; MCQ — Multiple Choice Question; AI — Artificial Intelligence; TLS — Transport Layer Security; UAT — User Acceptance Testing; WCAG — Web Content Accessibility Guidelines.

2. Document Overview

2.1 How to use this document

Sections 3 and 4 describe the architecture and design of SecureExam-AI. The architecture section explains the system boundary, components, technology choices, constraints, risks, requirement mapping, and security controls. The design section provides representative sequence flows, API/interface definitions, error handling, monitoring, UX considerations, and future work. The diagrams and tables are intended to support implementation and testing.

2.2 Related Documents

- Software Requirements Specification (SRS) — Online Exam Portal (SecureExam-AI), Version 1.0, dated 06-09-2026.
- Software Test Plan (STP) — Online Exam Portal (SecureExam-AI), Version 1.0, dated 16-09-2026.
- Requirements Traceability Matrix (RTM) contained in the SRS/STP.

3. Architecture

3.1 Goals & Constraints

The architecture is designed to support secure online examinations, separation of Student and Admin capabilities, reliable timed exam execution, browser-based proctoring, automatic evaluation, analytics, and report export.

- Use a React + Vite + Tailwind CSS frontend for responsive Student and Admin interfaces.
- Use an Express.js REST API on Node.js for authentication, exam, question, result, statistics, and export operations.
- Use MongoDB through Mongoose for persistent storage of users, exams, questions, attempts, and results.
- Protect authenticated APIs with JWT and role-aware access control; store passwords using bcryptjs.
- Keep correct answers out of student-facing responses before submission.
- Require fullscreen and camera access during an active proctored attempt.
- Support HTTPS/TLS in production and retain relevant attempt/violation logs for audit purposes.

Key constraints are the browser-based nature of proctoring, dependence on camera permission, stable internet connectivity, evergreen desktop/laptop browsers, and the project scope exclusions documented in the SRS.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Students	Simple exam interface, accurate timer, secure authentication, fair proctoring, reliable submission, and result visibility.
Admins	Exam/question lifecycle management, student-attempt review, analytics, and Excel report export.
Project Guide / Evaluator	Requirement coverage, correctness, security, testability, and acceptance readiness.
QA / Test Team	Traceable requirements, reproducible test flows, defect isolation, and measurable non-functional behaviour.
Developers	Modularity, maintainability, clear interfaces, secure API behaviour, and integration between frontend/backend modules.

3.3 Component (UML) Diagram

The logical component view below is derived from the SRS technology stack and the STP test-item/module structure.

Student UI React + Vite + Tailwind CSS	Admin UI React + Vite + Tailwind CSS	Browser AI Proctoring Camera + browser events
Express.js REST API **Authentication / RBAC** **Exam / Question / Result / Admin APIs**	**MongoDB + Mongoose** **Users • Exams • Questions** **Attempts • Results • Logs**	**ExcelJS / Recharts** **Excel result export** **Admin analytics**
JWT + bcryptjs Authentication & security	HTTPS/TLS Client-server communication	Axios / JSON Frontend ↔ REST API

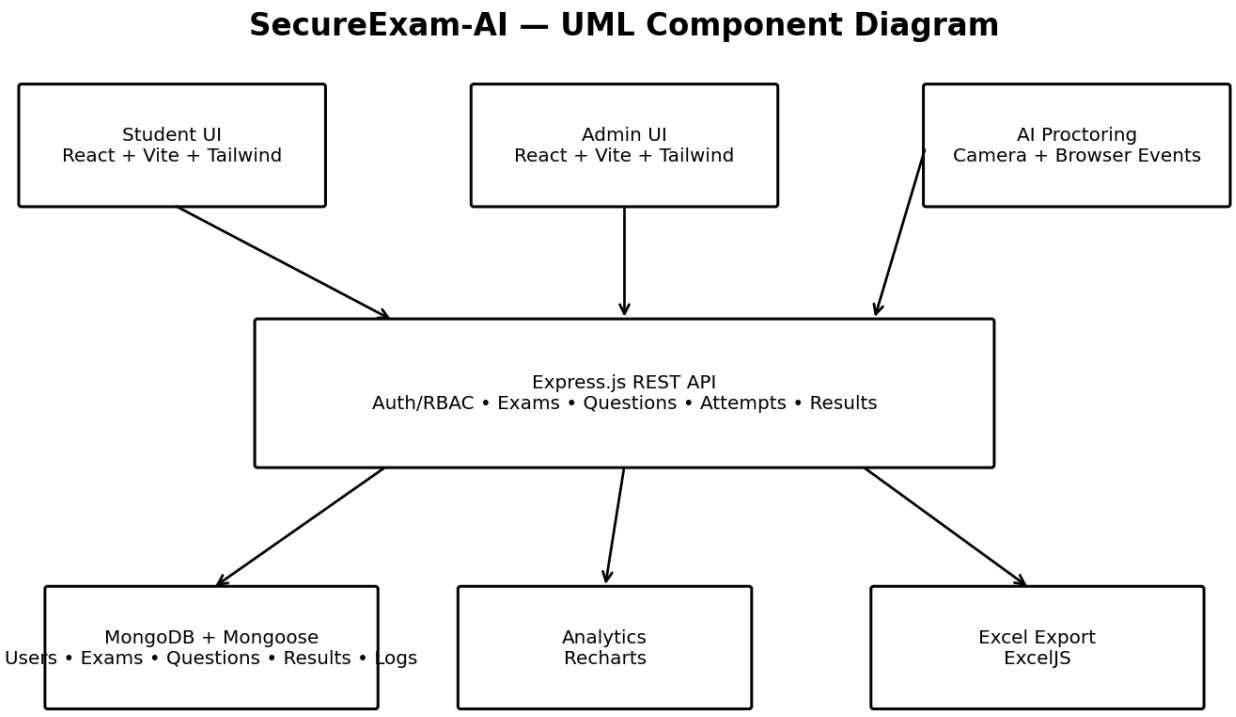


Figure 1. Logical component view of SecureExam-AI

3.4 Component Descriptions

Component	Responsibilities
Student Frontend	Registration/login, profile, available exams, exam attempt UI, timer, answers, submission, and result/history views.
Admin Frontend	Exam and question management, student/attempt review, analytics dashboard, and report export controls.
REST API / Backend	Business/API layer for authentication, authorization, exams, questions, attempts, results, statistics, and export.
Authentication & RBAC	JWT-based authenticated sessions, role-aware route protection, and bcryptjs password verification/storage.
AI Proctoring Module	Camera-based face monitoring, tab-switch detection, fullscreen-exit detection, violation counting, and threshold-based auto-submission.
MongoDB / Mongoose	Persistent storage for users, exams, questions, attempts, results, and relevant audit/violation information.
Analytics Module	Aggregates dashboard totals, performance statistics, and score distributions; uses Recharts on the frontend.
Excel Export Module	Generates downloadable result reports through ExcelJS; STP identifies /api/export/excel/:examId as the export endpoint.

3.5 Chosen Architecture Pattern and Rationale

A client-server architecture with logical separation between presentation, REST API/business processing, persistence, and browser proctoring responsibilities is used. The frontend is a React + Vite single-page application, the backend exposes an Express.js REST API, and MongoDB provides persistence. This structure matches the existing project organisation documented in the SRS and STP and keeps Student/Admin presentation concerns separate from server-side authorization, evaluation, and data operations.

3.6 Technology Stack & Data Stores

Layer / Area	Technology / Store	Purpose
Frontend	React + Vite + Tailwind CSS	Responsive Student/Admin UI and exam interface.
Backend	Node.js + Express.js	REST API and server-side application logic.
Database	MongoDB + Mongoose	Persistent application data.
Authentication	JWT + bcryptjs	Authenticated sessions, role claims, and password hashing.

HTTP Client	Axios	Frontend API calls.
Analytics	Recharts	Admin dashboard charts.
Report Generation	ExcelJS	Downloadable Excel result reports.
Proctoring	Browser camera + browser events	Face monitoring and tab/fullscreen event detection.

3.7 Risks & Mitigations

Risk	Mitigation / Design Response
Camera permission unavailable	Require camera permission for proctored attempts; maintain a verified test machine and document fallback/mock camera handling for testing.
Browser-specific tab/fullscreen behaviour	Validate on Chrome, Edge, and Firefox and record browser-specific behaviour as known issues where necessary.
Unstable backend/database environment	Maintain a stable test environment and a local MongoDB fallback as described in the STP.
Timer / auto-submit race conditions	Use controlled timers in test environments and explicit waits in automated tests.
Unauthorized access to Admin operations	JWT role claims and server-side RBAC checks protect Admin-only routes.
Exposure of answer keys	Student-facing pre-submission responses must exclude correct-answer fields.

3.8 Traceability to Requirements

Requirement	Architectural Element
OEP-F-002 Login & JWT issuance	Authentication layer / REST API
OEP-F-010 Create exam	Admin UI + Exam Management API + Exam data model
OEP-F-013 Add question	Admin UI + Question Management API + Question data model
OEP-F-030..034 AI proctoring	Browser AI Proctoring Module + Attempt Module
OEP-F-040..043 Evaluation, history, analytics, export	Result Module + Analytics Module + Excel Export Module
OEP-NF-001 Response time	Frontend/API design + performance testing
PRJ-SR-002 Password hashing	Authentication layer + bcryptjs
PRJ-SR-004 No answer-key exposure	Question API response design
PRJ-SR-005 Append-only violation audit log	Proctoring/DB audit logging

3.9 Security Architecture

Security is implemented as a set of controls distributed across authentication, API authorization, data handling, communications, and proctoring audit logging. The SRS requires TLS 1.2+ in production, bcryptjs password hashing, short-lived signed JWTs with role claims, protection against answer-key exposure, and timestamped append-only violation events.

Security Concern	Architecture Control
Authentication	Email/password authentication with bcryptjs password hashing and signed JWTs.
Authorization	Role-aware JWT sessions and Admin-only route protection; unauthorized Student access should return 403.
Confidentiality in transit	HTTPS/TLS for production client-server communication.
Question/answer confidentiality	Correct answers are excluded from student-facing pre-submission API responses.
Proctoring auditability	Violation events are timestamped and retained as an audit record per attempt.
Token integrity	Expired or tampered signed JWTs are rejected.

Threat-model coverage is therefore focused on spoofing/unauthorized access, tampering with tokens or requests, information disclosure of credentials/answer keys, and misuse of protected routes. The detailed security requirements and corresponding test cases are maintained in the SRS/STP.

4. Design

4.1 Design Overview

The design follows the component boundaries established in the architecture. Student and Admin interfaces communicate with the Express REST API using JSON. The API validates authentication and authorization before protected operations. Mongoose provides access to MongoDB. During an exam attempt, the browser UI coordinates the timer and proctoring events; the result flow evaluates objective answers on submission and publishes the resulting score/history to the student. Admin analytics and Excel export consume stored attempt/result information.

4.2 UML Sequence Diagrams

Two representative flows are provided below. They are derived from the functional flows and test cases already defined in the SRS/STP.

4.2.1 Student Exam Attempt and Submission

Step	Actor / Component	Interaction	Outcome
1	Student	Selects an available exam and starts attempt.	Attempt screen opens; fullscreen/camera permissions are requested.
2	Student UI	Starts countdown and displays questions.	Student answers within the configured duration.
3	Proctoring Module	Monitors camera/browser events.	Face monitoring and tab/fullscreen violations are recorded/countable.
4	Student / Timer	Submits manually or timer reaches 00:00.	Attempt is submitted automatically or manually.
5	REST API / Result Module	Receives submission and evaluates objective answers.	Score/result is generated.
6	Database	Persists attempt/result and relevant violation data.	Result becomes available for history/analytics.
7	Student UI	Requests/opens result history.	Student sees the published result.

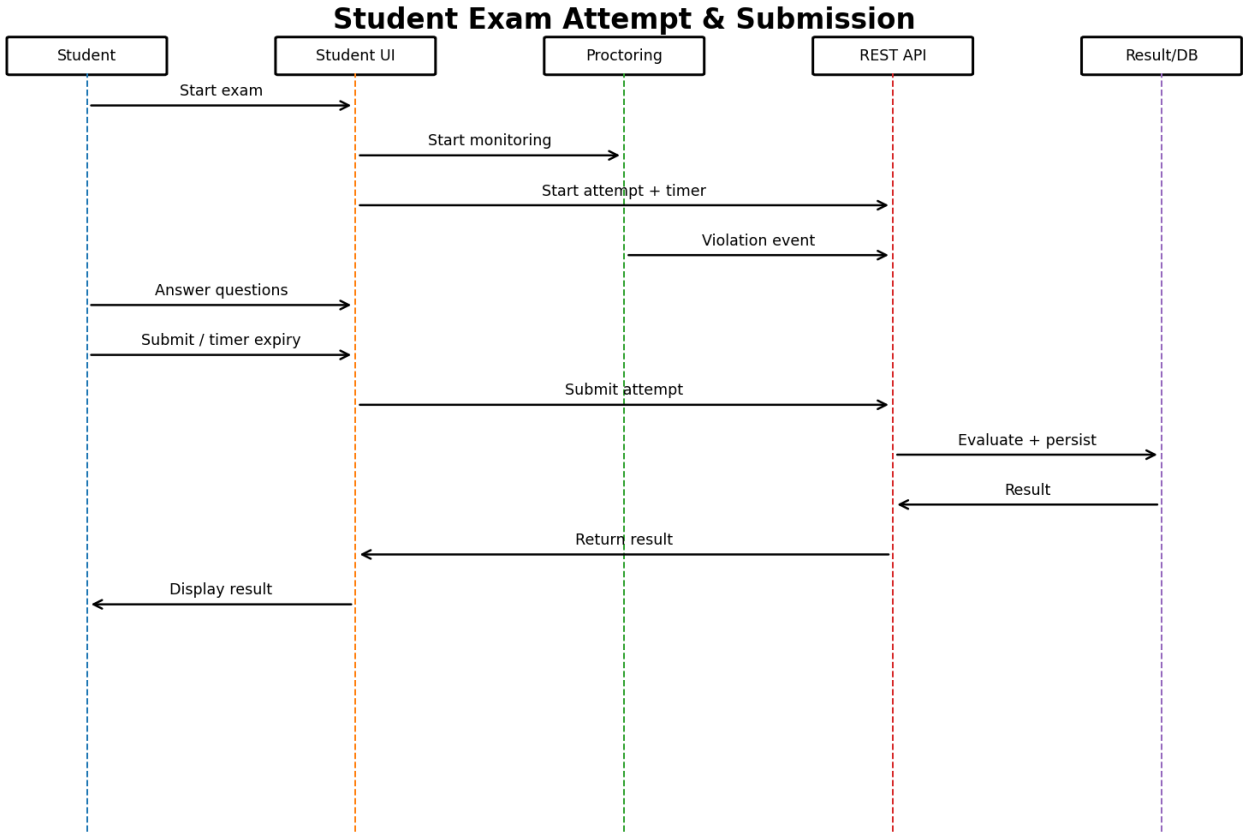


Figure 2. Sequence flow — Student exam attempt, proctoring and submission

4.2.2 Admin Creates Exam and Adds Questions

Step	Actor / Component	Interaction	Outcome
1	Admin	Logs in with Admin credentials.	JWT-authenticated Admin session is established.
2	Admin UI	Opens Manage Exams and enters title, duration, schedule.	Exam details are prepared for submission.
3	REST API	Validates Admin authorization and exam data.	Create operation is accepted for processing.
4	MongoDB / Mongoose	Stores the new exam.	Exam appears in Admin exam list.
5	Admin UI	Opens question manager and enters question, options and correct answer.	Question data is prepared.
6	REST API + MongoDB	Validates and stores the question linked to the exam.	Question is persisted and associated with the exam.
7	Admin UI	Refreshes exam/question list.	Admin can review the updated exam and question set.

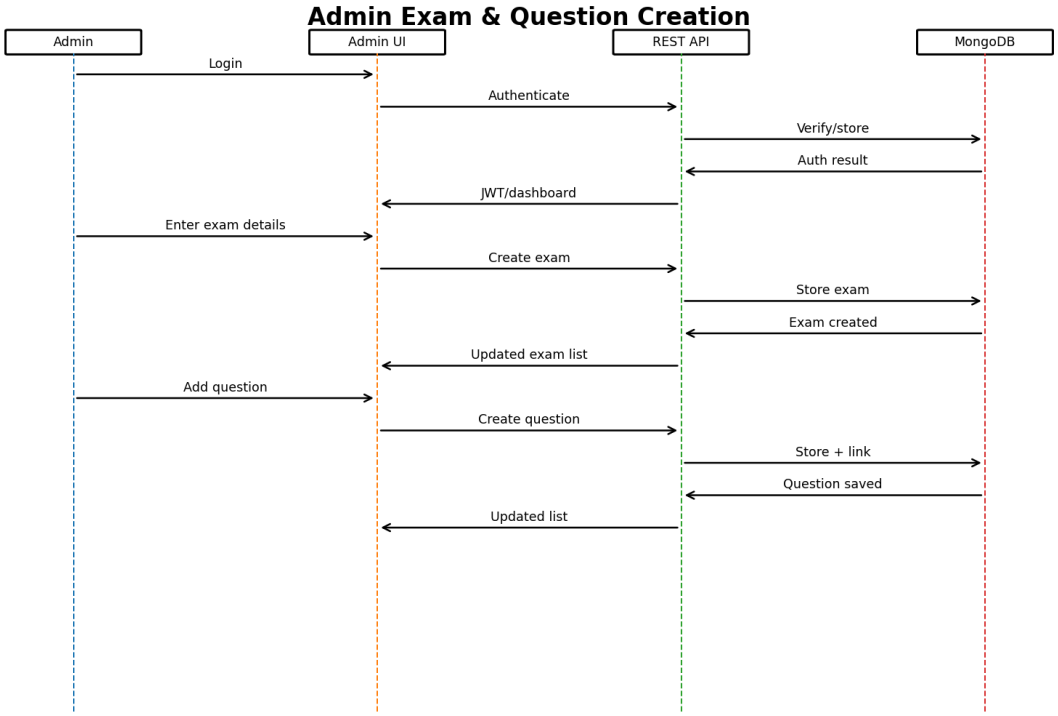


Figure 3. Sequence flow — Admin exam and question creation

4.3 API Design

The backend is an Express.js REST API using JSON. The STP explicitly identifies authentication, exams, questions, results, admin statistics, and export as API areas. The following interfaces document the project-supported interactions without introducing endpoint names that are not specified in the source documents.

Interface	Method / Input	Output / Behaviour	Security
Authentication interface	Email + password; JSON request	Successful authentication issues a signed JWT; invalid credentials are rejected.	JWT + bcryptjs; role-aware session.
Exam management interface	Admin exam details: title, duration, schedule	Creates/updates/deletes exams according to business rules.	Admin RBAC required.
Question management interface	Question text, options, correct answer	Stores and links questions to an exam; edit/delete supported.	Admin RBAC; answer key protected from student pre-submission responses.
Admin statistics	GET /api/admin/stats	Returns dashboard statistics used by Admin analytics.	Admin authorization required.
Excel result export	GET /api/export/excel/:examId	Downloads an Excel report containing student name, email, exam, score, percentage, and submission date.	Admin authorization required.
Result interface	Submitted objective answers	Evaluates answers and publishes result/history data.	Authenticated Student/Admin access as applicable.

4.4 Error Handling, Logging & Monitoring

Error handling should provide clear, role-appropriate responses while avoiding disclosure of sensitive information. Authentication and authorization failures are represented by rejection of invalid credentials/tokens and protected-route access; the STP expects 401 for missing/invalid tokens and 403 for protected Admin routes accessed by a Student token.

- Reject invalid or expired/tampered JWTs and do not continue protected processing.
- Return 403 for a valid Student token attempting an Admin-only operation.
- Prevent deletion of an exam when active attempts exist, consistent with the SRS requirement.
- Keep plaintext passwords and other sensitive credentials out of logs.
- Record timestamped proctoring violation events for each attempt.
- Monitor availability, response time, test execution results, defect trends, and requirement coverage as described in the STP.

4.5 UX Design

The user interface is a responsive React + Tailwind CSS web interface with separate Student and Admin dashboards. The Student exam screen is designed to minimise distractions and includes a live countdown timer. The SRS specifies usability on common screen sizes including 1366×768 and above and calls for WCAG 2.1 AA colour-contrast guidance where applicable.

- Student flow: Register/Login → Available Exams → Exam Details/Start → Fullscreen Timed Attempt → Submit/Auto-submit → Results/History.
- Admin flow: Login → Manage Exams → Manage Questions → Review Students/Attempts → Analytics → Excel Export.
- Use clear validation and error messages for authentication, exam operations, and submission failures.
- Keep proctoring status/violations visible without exposing implementation details or sensitive information.
- Maintain responsive layouts suitable for the documented desktop/laptop test environment.

4.6 Open Issues & Next Steps

The following items remain implementation/validation considerations based on the current SRS and STP rather than additional product scope:

- Complete and verify all planned implementation modules against the RTM.
- Execute the full STP test suite and update RTM statuses from Not Run to Pass/Fail as evidence becomes available.
- Validate AI proctoring behaviour consistently across Chrome, Edge, and Firefox.
- Complete performance validation for the ≤2 second 90th-percentile response target.
- Complete usability/accessibility checks at 1366×768 and above.
- Verify production TLS configuration, JWT expiry/tamper rejection, answer-key protection, and audit-log behaviour.
- Prepare the final Test Summary Report and evaluator sign-off after exit criteria are satisfied.

5. Appendices

5.1 Glossary

Term	Meaning
AI	Artificial Intelligence; used here for browser-based proctoring logic.
API	Application Programming Interface.
JWT	JSON Web Token used for signed authenticated sessions.
RBAC	Role-Based Access Control.
REST	Representational State Transfer style used for backend APIs.
Mongoose	MongoDB Object Document Mapper used by the backend.
MCQ	Multiple Choice Question.
RTM	Requirements Traceability Matrix.
SRS	Software Requirements Specification.
STP	Software Test Plan.
SAD	Software Architecture and Design Specification.
TLS	Transport Layer Security for protected client-server communication.

5.2 References

- Software Requirements Specification (SRS), Online Exam Portal (SecureExam-AI), v1.0, 06-09-2026.
- Software Test Plan (STP), Online Exam Portal (SecureExam-AI), v1.0, 16-09-2026.
- Provided Software Architecture and Design Specification template (ATM System example).

5.3 Tools

Tool / Technology	Use in the Project / Documentation
React + Vite + Tailwind CSS	Frontend implementation.
Node.js + Express.js	Backend REST API.
MongoDB + Mongoose	Database and data access.
ExcelJS	Excel report generation.
Recharts	Admin analytics visualisation.
Axios	Frontend API communication.
Postman	REST API testing.
Jest / Mocha + Supertest	Backend unit/integration testing.
React Testing Library	Frontend component testing.
Cypress / Selenium WebDriver	End-to-end UI automation.
JMeter	Performance/load testing.
MongoDB Compass	Database inspection.
GitHub Issues / Jira	Defect and test-case tracking.
Lighthouse / axe DevTools	Usability and accessibility checks.